泰坦尼克号生存预测 —— 学习笔记

使用 PyTorch 自主构建二分类模型，根据乘客个人信息预测其是否幸存

一、数据清洗和特征处理方法

数据集是 `train.csv`，一共 891 条乘客记录，每条有 12 个字段

1. 缺失值处理

先数了一下每一列缺了多少：

- Age（年龄）缺 177 条（19.9%），缺得不算多，用中位数填 选中位数不选平均数，是因为年龄里有个别很大的值，平均数容易被这些极端值拉偏
- Embarked（登船港口）只缺 2 条，用众数填最省事，这点数据影响不了什么
- Cabin（客舱号）缺了 687 条（77%），剩下那点信息也拼不出什么规律，这一列直接删掉

这里有个坑我一开始踩了 我最开始是先把缺失值全填完，再划分训练集和测试集 后来看题目 3.2 才发现这样是错的，填年龄用的中位数是把测试集的年龄也算进去算出来的，相当于测试集的信息漏进了训练集（数据泄露）

改法是把顺序倒过来：先划分，再只用训练集算中位数、众数、均值和标准差，然后测试集和新样本都套这一套参数 代码里拆成了 `fit_preprocessor()` 和 `transform()` 两个函数，前者只在训练集上调用

2. 特征工程

- 性别 Sex：模型只认数字，所以 male 映射成 0，female 映射成 1 这是最强的特征
- 登船港口 Embarked：C / Q / S 三个值，用独热编码 如果直接映射成 0/1/2，模型会以为这三个港口之间有大小关系，其实没有 编码时按值比较 `(Embarked == 'Q') / (Embarked == 'S')`直接生成这两列，把 C 当基准，所以最后实际只有 Embarked_Q 和 Embarked_S 两列，不是三列 这样做一是避免三列加起来恒等于 1 造成共线性，二是少一个特征 没用 pd.get_dummies(drop_first=True) 是因为它按这一批数据里出现过哪些港口决定生成哪几列，只喂一行时会把唯一的类别丢掉；按值比较的结果和这一批里有几个港口无关
- 姓名 Name：原始姓名本身没什么用，本来想直接删 但姓名里带着称谓（Mr / Mrs / Miss / Master 这些），而称谓其实间接暴露了乘客的性别、年龄和阶级，是个挺强的隐藏特征 所以用正则表达式把称谓抠出来，归成 5

类：Mr=1、Miss=2、Mrs=3、Master=4，剩下少见的（Lady、Dr、Rev 这些）统一归成 Rare=5

- SibSp + Parch：这两个分别是同行的兄弟姐妹/配偶数和父母/子女数 把它们加起来再 +1（算上自己）造一个新特征 FamilySize（家庭人数） 因为单独看这两个数不太容易看出“这是一家人”以及家庭规模大小，而一家人的生存率明显和独身乘客不一样
- Age 和 Fare 标准化：年龄范围是 0~80，票价范围是 0~512，量级差太多了直接喂进网络的话，票价数值大，容易在梯度里独占风头 所以用 `(x - 均值) / 标准差` 把两个都拉到差不多的范围 用的是训练集算出来的均值和标准差

处理完一共 10 个特征，顺序固定成：

```
Pclass, Sex, Age, SibSp, Parch, Fare, Embarked_Q, Embarked_S, Title, FamilySize
```

这个顺序在代码里用一个 `FEATURE_COLUMNS` 列表写死，训练和预测都从它取，这样列一定对得上

二、模型结构、损失函数、优化器及主要超参数

1. 模型结构

用 `torch.nn.Module` 自己搭的全连接神经网络（MLP），结构是：

```
输入层(10) → 全连接 32 → ReLU → Dropout 0.2
          → 全连接 16 → ReLU → Dropout 0.2
          → 全连接 1  → Sigmoid
```

- 输入层 10 个神经元，对应上面那 10 个特征；输出层 1 个神经元，过 Sigmoid 变成 0~1 之间的数，可以当生存概率看 大于 0.5 判成幸存（1），否则遇难（0）
- 两个隐藏层，分别 32 和 16 个神经元，激活函数 ReLU 一开始我只写了一层 16 个神经元，感觉容量不太够，训练准确率一直上不去，后来加宽了一层
- 中间加了 Dropout（0.2），每次训练随机让 20% 的神经元失效 因为训练集准确率一直比测试集高，说明有点过拟合，让一部分神经元随机失效能逼着网络不要只依赖某几个神经元

2. 损失函数

用 `nn.BCELoss()`（二分类交叉熵） 因为输出已经过 Sigmoid 落在 0~1 之间，标签是 0 或 1，正好配套 标签要转成 float32 并用 `.view(-1, 1)` 拉成一列，不然形状和输出对不上会报错

3. 优化器

Adam，学习率 lr = 0.001，weight_decay = 1e-4

学习率这里改过一次：一开始设的 0.01，损失下降是快，但测试集准确率抖得厉害，像心电图一样 降到 0.001 之后就平稳多了

`weight_decay` 这个参数是 Adam 优化器的，不是损失函数的 它相当于在每次更新参数的时候把权重往 0 拉一点（相当于 L2 正则化），能压一压过拟合 我一开始以为是加在损失函数上的，后来查了资料才知道是优化器的参数

4. 主要超参数

超参数	取值
训练轮次 Epochs	150
批次大小 Batch Size	32
学习率 lr	0.001
weight_decay	1e-4
Dropout	0.2

三、数据划分方式、随机种子和实验配置

1. 数据划分

用 `train_test_split` 按 8 : 2 划分，训练集 712 条，测试集 179 条

本来还想再切一个验证集出来，但一共就 891 条数据，再切一块测试集就太小了，所以最后还是按题目说的分成训练集和测试集两块

顺序是先划分，再在训练集上算预处理参数，理由写在第一节里了

2. 随机种子

设了 `SEED = 42` 为了可复现，把能设的随机种子都设了一遍：

- `random.seed(SEED)` —— Python 自带的随机数
- `np.random.seed(SEED)` —— numpy 的
- `torch.manual_seed(SEED)` —— 影响模型参数的初始化
- `torch.cuda.manual_seed_all(SEED)` —— 这一行是照着教程顺手写上的 我装的是 GPU 版的 PyTorch，但这个作业没让它上显卡，模型和数据都没 `.to(device)`，默认就在 CPU 上，所以这一行其实没起作用，只是留着以后用

另外两处随机性也固定住了：

- `train_test_split(..., random_state=SEED)` —— 固定数据怎么切
- `DataLoader(..., shuffle=True, generator=torch.Generator().manual_seed(SEED))` —— 固定每一轮打乱样本的顺序 这个如果不设，每轮取到的批次不一样，结果也会飘

验证过：同样的代码连着跑两遍，每一轮的损失和准确率到小数点后四位完全一样

3. 实验配置

代码拆成几个文件：`data_preprocessing.py`（数据清洗和特征工程）、`model.py`（网络结构）、`train.py`（训练、保存、画图）、`predict.py`（新样本预测）、`main.py`（入口，把前面几件事串起来）

路径我写的是相对项目根目录的（`data/train.csv`、`save/titanic_model.pth` 这种），所以命令一定得在项目根目录下敲 一开始我在 `src` 文件夹里敲 `python main.py`，一直报文件找不到，后来才反应过来路径是从哪儿开始算的

训练完会保存两样东西：

- `save/titanic_model.pth` —— 模型参数
- `save/preprocess_params.json` —— 预处理参数（年龄中位数、港口众数、均值、标准差、称谓映射表、特征列顺序）

第二样是关键 预测新样本的时候不能再手写一遍特征工程，那样很容易和训练时对不上 现在是直接把 json 读回来，调用训练时用的同一个 `transform()` 函数，保证新样本和训练数据是用同一套方式处理的

四、训练过程中遇到的问题及解决方法

1. 环境冲突

在终端直接敲 `python train.py` 报错，说找不到 torch 排查发现默认用的是 base 环境，base 里没装 torch 解决方法：先 `conda activate pytorch` 切到装好 torch 的环境再跑

2. 张量类型转换错误

数据传进 PyTorch 的时候报错，说数据类型不对 原因是 pandas 读出来的列有 int64 也有 float64，混在一起转 tensor 会出问题 解决方法：统一 `.astype(np.float32)` 转一遍

3. 模型维度不匹配

跑预测的时候报 `size mismatch for fc1.weight`，意思是模型输入的维度和喂进去的数据维度不一样 排查发现训练时输入是 10 个特征，预测的时候我手写的字典

只有 9 个，少了一个 这次重写我干脆把特征列名用一个列表写死，训练和预测都从这个列表取，从根上避免这类问题

4. 底层库损坏

中途遇到很奇怪的报错，说 `torch` 没有 `version` 这个属性，但 `import torch` 又没报错 排查发现是 PyTorch 装得不完整，底层文件缺了 解决方法：先卸载，再换用国内镜像源重新完整装一遍

5. 数据泄露（这个是最值得说的）

一开始我的预处理顺序是“先填缺失值 → 再划分训练集测试集”，写的时候没觉得有问题 后来看题目 3.2 里那句“预处理参数只能根据训练数据确定，测试数据和新样本必须复用同一套处理方式”，才反应过来：填年龄用的中位数是拿全部 891 条算出来的，里面包含了测试集的年龄 这就是数据泄露

改法：把预处理拆成 `fit_preprocessor()`（只在训练集上算参数）和 `transform()`（用参数处理数据），先划分再 fit，测试集和新样本都调 `transform`

改完之后我特意把两种做法都跑了一遍做对比：

- 在全部 891 条上算参数（泄露版）：最好测试集准确率 0.8212
- 只在训练集 712 条上算参数（正确版）：最好测试集准确率 0.8156

差了 0.56 个百分点 泄露版高出来的这一点点，就是模型提前“看到”测试集信息换来的，是虚的 所以虽然数字变低了，0.8156 这个才是可信的、能反映真实水平的数字

6. 画图卡住

训练完调用 `plt.show()` 之后脚本一直不结束，要手动关掉窗口 解决方法：改成只 `plt.savefig()` 存图，并且用 `matplotlib.use('Agg')` 指定不弹窗的后端，最后加上 `plt.close()` 把图关掉

五、测试集准确率、实验结果分析和心得体会

1. 测试集准确率

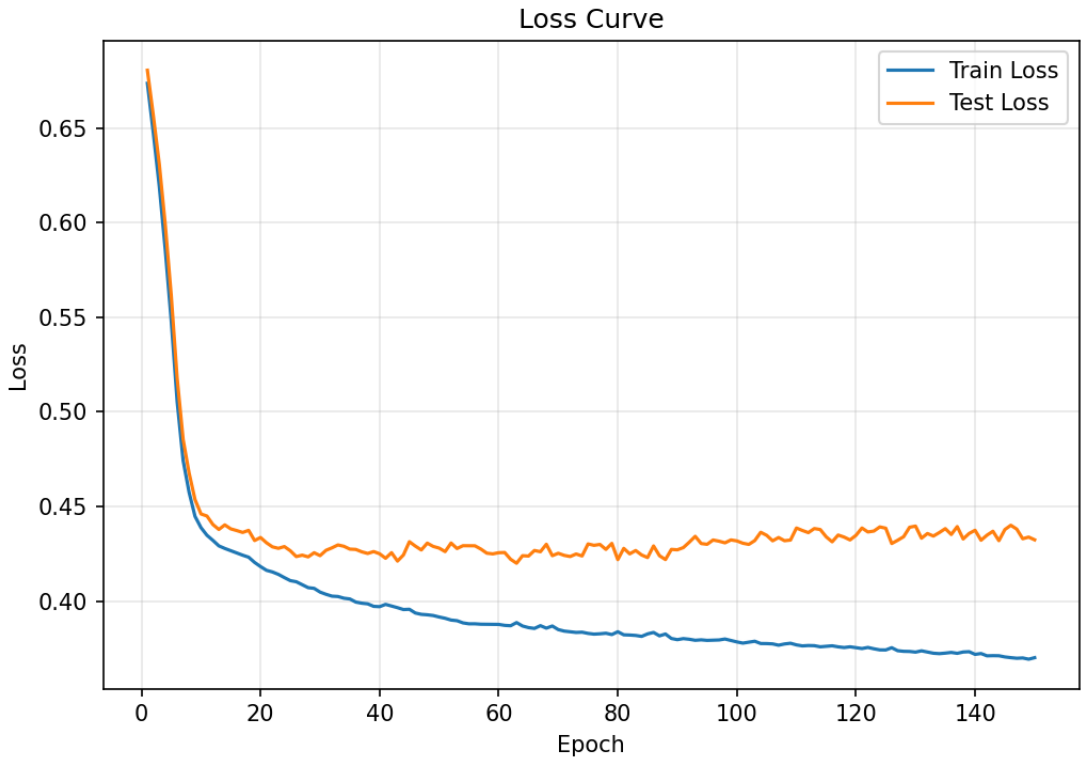
训练 150 轮之后，结果如下：

指标	数值
训练集准确率	0.8553（85.53%）
测试集准确率	0.8156（81.56%）
训练集损失	0.3700
测试集损失	0.4324

测试集准确率最高的一轮是第 67 轮，也是 0.8156，保存的就是这一轮的模型

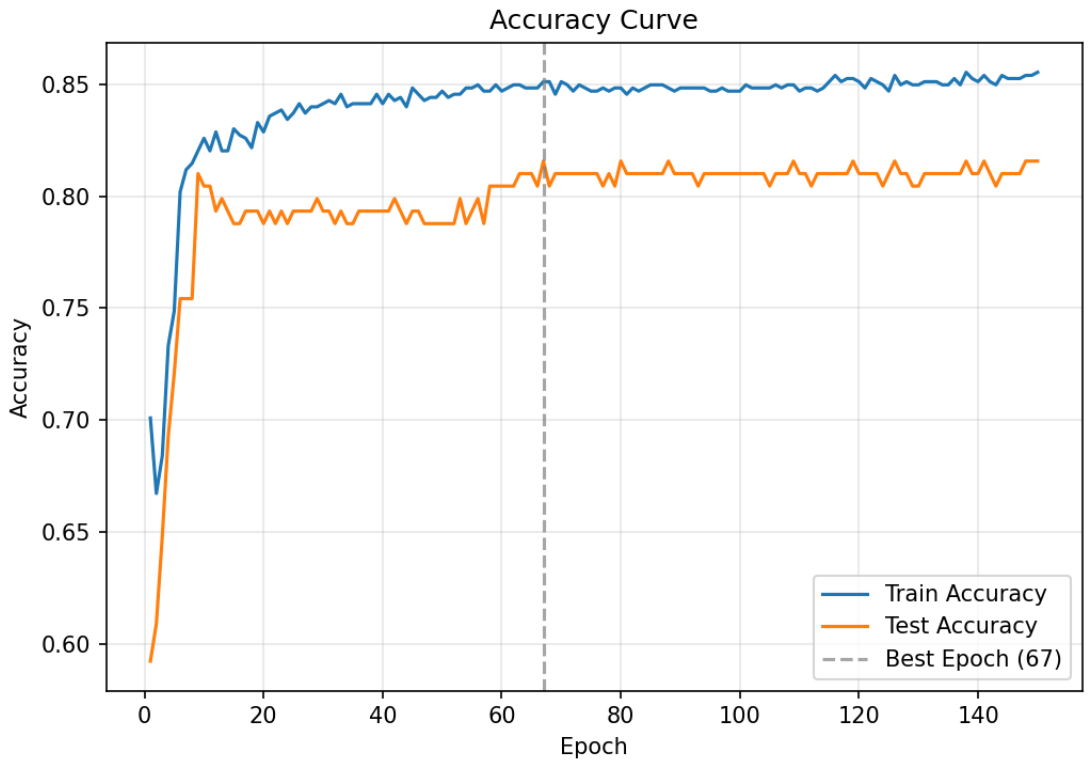
2. 曲线分析

图 1 是模型损失趋势图（训练损失与测试损失随训练轮次的变化）：



模型损失趋势图

图 2 是训练集与测试集准确率图（虚线标出测试集准确率最高的第 67 轮）：



训练集与测试集准确率图

下面再把这两张图的大致走势说一下

从损失曲线看：训练损失一直在往下掉，从 0.67 掉到 0.37；测试损失前 20 轮降得很快，从 0.68 掉到 0.43 左右，但 20 轮之后基本就不再降了，后面甚至慢慢往上抬 两条线在第 20 轮附近分开，之后越拉越远

从准确率曲线看：训练集准确率从 0.70 左右一路涨到 0.855；测试集准确率涨到 0.80 左右之后，就在 0.79 ~ 0.82 之间来回晃，基本不再提高 两条线大概差了 4 个百分点

这两个现象都说明出现了过拟合：模型把训练集里的细节（甚至噪声）也一起背下来了，所以在训练集上表现越来越好，但在没见过的测试集上就没什么进步 不过差距只有 4 个百分点，不算特别严重，属于轻度过拟合

3. 心得体会

这是我自己第一次完整做一个从数据到预测的机器学习项目，最大的收获是很多问题不在模型上，而在数据上

① 数据泄露这件事给我的印象最深 一开始我完全没意识到“先填缺失值再划分”是错的，从直觉上想，先把数据收拾干净再划分，多自然啊 是看了题目要求回头检

查才发现的 这让我明白，数据处理的顺序本身就是一个会影响结果正确性的东西，不是随便写写就行 所以后来我干脆把预处理拆成 fit 和 transform 两个函数，从代码结构上把这件事固定住

② 第一次从曲线上真真切切看到过拟合 以前只知道“过拟合”是个概念，这次是眼看着两条线从第 20 轮开始分叉：训练损失还在降，测试损失却已经到底了 这比背书上的定义印象深多了

③ 关于调参 我把学习率从 0.01 改成 0.001，又加了 Dropout 和 weight_decay，模型确实稳了一些，但准确率提高得有限（大概两三个百分点） 我原来的想法是“多加几层、多加几个神经元准确率就能上去”，试下来发现不是这么回事，891 条数据太少了，模型搞复杂了反而更容易过拟合 数据量和特征的质量，可能比模型结构更重要

④ 关于自己动手写代码 这次有几个部分（比如画图）我参考了吴恩达老师课上讲的内容，自己动手敲了一遍，但有些地方写得不对，跑起来就报错，又反复查资料改了好几遍才对 这个过程虽然慢，但比直接抄一遍有用得多，报错的时候被迫去想“这一步到底在干什么”，反而把原理弄明白了 现在回头看，能自己把整个流程从头到尾跑通，比记住某几行代码重要得多

⑤ 还可以继续改进的地方：

- 数据划分可以按姓氏分组 这次看数据时我发现有些乘客是同一家人（姓氏相同），随机划分的话可能出现哥哥在训练集、弟弟在测试集的情况 模型在训练集里“见过”哥哥，再去测弟弟，结果就会偏乐观 以后可以试试按姓氏整组划分，把一家人整个分到同一边，不拆开
- 891 条数据确实太少 可以用交叉验证代替单次划分，结果会更稳一些；或者想办法做点数据增强
- 特征还可以再挖 比如 Ticket（船票号）的前缀、Cabin（客舱号）的甲板层，都可能藏着信息